

## 10.3.1 项目总览与仿真环境搭建

### 课程信息

- 环境：Windows 10、WSL2 Ubuntu 24.04、ROS 2 Jazzy、Gazebo Harmonic。
- 本节目标：完成项目架构认识、Gazebo 麦轮仿真、传感器接入和全向遥控验证。
- 主路径：Gazebo UI。
- 兜底路径：Gazebo headless 或内置轻量 2D 仿真器。

路径约定：本讲义假设项目位于 `~/mecamind_ros2`。如果你把项目放在其他位置，请把命令中的路径替换为你的实际路径。获取项目的方式：把老师提供的 `mecamind_ros2` 项目压缩包解压到家目录，或按课堂说明克隆仓库后进入项目根目录。

本节不要求完成 SLAM、Nav2、YOLO 和语音任务。它们会在后续五次课中逐步接入。

### 1. 本节学习目标

完成本节后，应当能够：

1. 说清仿真层、导航层、感知层、控制层和任务层的职责。
2. 解释普通差速底盘与麦克纳姆底盘的主要区别。
3. 启动 Gazebo 三房间世界并看到机器人。
4. 检查 LiDAR、IMU、Camera、Odom 和 TF。
5. 让机器人完成前后、横移、旋转和组合运动。
6. 在 GUI 不可用时切换为 headless 或轻量仿真。
7. 使用自动脚本完成 CP1 验收。

### 2. 本节重点内容

#### 2.1 建立 ROS 2 工程的基本认识

本项目不是一个单独运行的 Python 脚本，而是由多个 ROS 2 包、节点和通信接口组成的工作空间。学习本项目时，需要先建立以下基本概念：

- **工作空间 (Workspace)**：保存多个 ROS 2 包的工程目录。本项目根目录就是一个工作空间，源码位于 `src/`，执行 `colcon build` 后会生成 `build/`、`install/` 和 `log/`。
- **包 (Package)**：ROS 2 中组织代码和资源的基本单位。一个包通常只承担一类职责，例如机器人描述、Gazebo 仿真或统一启动。
- **节点 (Node)**：运行时执行具体任务的程序单元。例如传感器桥接节点负责转发消息，`robot_state_publisher` 负责发布机器人坐标关系。一个操作系统进程可以包含一个或多个节点。
- **话题 (Topic)**：节点之间持续传输数据的通道。发布者把消息发送到话题，订阅者从话题接收消息，双方不需要知道彼此的进程地址。
- **消息类型 (Message Type)**：规定数据的结构。例如 `/cmd_vel` 使用 `geometry_msgs/msg/Twist`，其中 `linear` 表示线速度，`angular` 表示角速度。
- **服务与动作 (Service/Action)**：服务适合一次请求、一次响应；动作适合耗时任务并能够反馈进度。第一节

主要使用话题，后续导航任务会使用 Nav2 Action。

- **参数 (Parameter)**：节点运行时使用的配置，例如传感器频率、地图文件路径和最大速度。
- **启动文件 (Launch)**：一次启动多个节点，并统一设置参数、话题重映射和环境变量。本项目通过 launch 启动完整仿真，而不是要求学员逐个启动节点。

ROS 2 底层通常使用 DDS 完成节点发现和消息传输。处于同一网络、相同 `ROS_DOMAIN_ID` 的节点可以自动发现彼此；Domain 不一致时，即使程序都在运行，`ros2 topic list` 也可能看不到对方。测试残留进程会造成节点重名、话题数据混杂或端口资源冲突，因此每次重新测试前应先确认旧仿真已经退出。

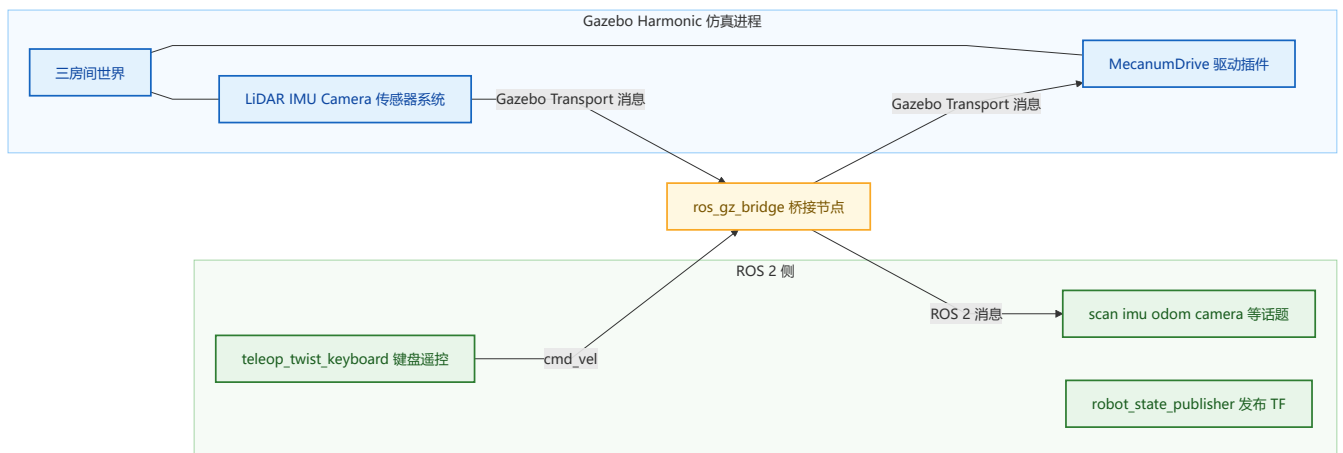
## 2.2 理解项目分层与数据流

项目按照仿真、导航、感知、控制和任务进行分层。分层的目的不是增加文件数量，而是让每一层只依赖稳定接口：

- 仿真层或真机驱动负责产生传感器数据，并执行速度指令。
- 导航层根据地图、定位和障碍物计算运动速度。
- 感知层从相机图像中识别目标，并输出目标位置。
- 控制层处理遥控、导航、视觉跟随和急停之间的速度竞争。
- 任务层把“去客厅”“开始巡逻”等高级指令拆分为导航和控制任务。

需要重点理解一条完整数据链路：Gazebo 传感器产生数据，经桥接转换为 ROS 2 消息，算法节点订阅消息并计算结果，控制节点发布 `/cmd_vel`，桥接再把速度命令送回 Gazebo 驱动机器人。任何一段断开，机器人都不能形成完整闭环。

本节课实际搭建的数据链路如下：



传感器数据自左向右流入 ROS 2，速度命令自右向左返回 Gazebo。图中黄色的桥接节点是唯一的转换通道，排错时应先确认它两侧各自是否有数据。

## 2.3 理解 Gazebo 仿真的组成

Gazebo 不只是一个三维显示窗口，它同时包含物理计算、传感器模拟和可视化：

- **World** 定义地面、墙体、光照、障碍物、物理引擎参数以及放入世界中的机器人。
- **Model** 定义机器人由哪些刚体和关节组成，以及每部分的质量、惯量、碰撞体和外观。
- **Link** 表示一个刚体，例如车身、车轮、LiDAR 和相机。
- **Joint** 描述两个 Link 之间如何连接，例如车轮与车身之间的旋转关节。

- **Sensor** 根据仿真环境生成激光、IMU 和图像数据。
- **System Plugin** 在每个仿真周期执行功能，例如麦克纳姆轮驱动、传感器更新和场景状态发布。

Gazebo Transport 与 ROS 2 是两套不同的通信系统。Gazebo 中已经出现 `/scan`，并不代表 ROS 2 一定能看到 `/scan`。`ros_gz_bridge` 必须知道两端的话题名、消息类型和桥接方向，才能在两套系统之间转换数据。

## 2.4 掌握麦克纳姆轮全向运动原理

普通差速底盘通常只能直接前后运动和旋转，不能在保持车头方向不变的情况下横向平移。麦克纳姆轮圆周上安装了倾斜滚子，轮子转动时，接触地面的作用力会分解为车体前后方向和左右方向两个分量。

四个轮子的滚子方向成对镜像安装。从机器人正上方俯视，本项目采用常见的 X 型安装，四个轮子接地滚子的方向连起来近似一个 X：

```

      车头方向 ↑
    [\] 前左      前右 [/]
      车体
    [/] 后左      后右 [\]
```

图中 `\` 和 `/` 表示俯视时接地滚子的倾斜方向。控制器通过组合四个车轮的转速，使某些方向的分力相互抵消，另一些方向的分力相互叠加，从而实现：

- 四轮配合产生相同的前向合力，机器人前进或后退。
- 四轮横向分力同向叠加，前向分力相互抵消，机器人左右横移。
- 左右两侧产生相反方向的力，形成绕车体中心的转矩，机器人原地旋转。
- 同时给出前向、横向和旋转速度，机器人执行斜向平移或边移动边转向。

ROS 2 不直接向每个轮子发送速度，而是通过 `/cmd_vel` 表达车体期望速度。麦轮驱动插件再根据底盘尺寸和轮半径，把车体速度分解为四个轮子的角速度。

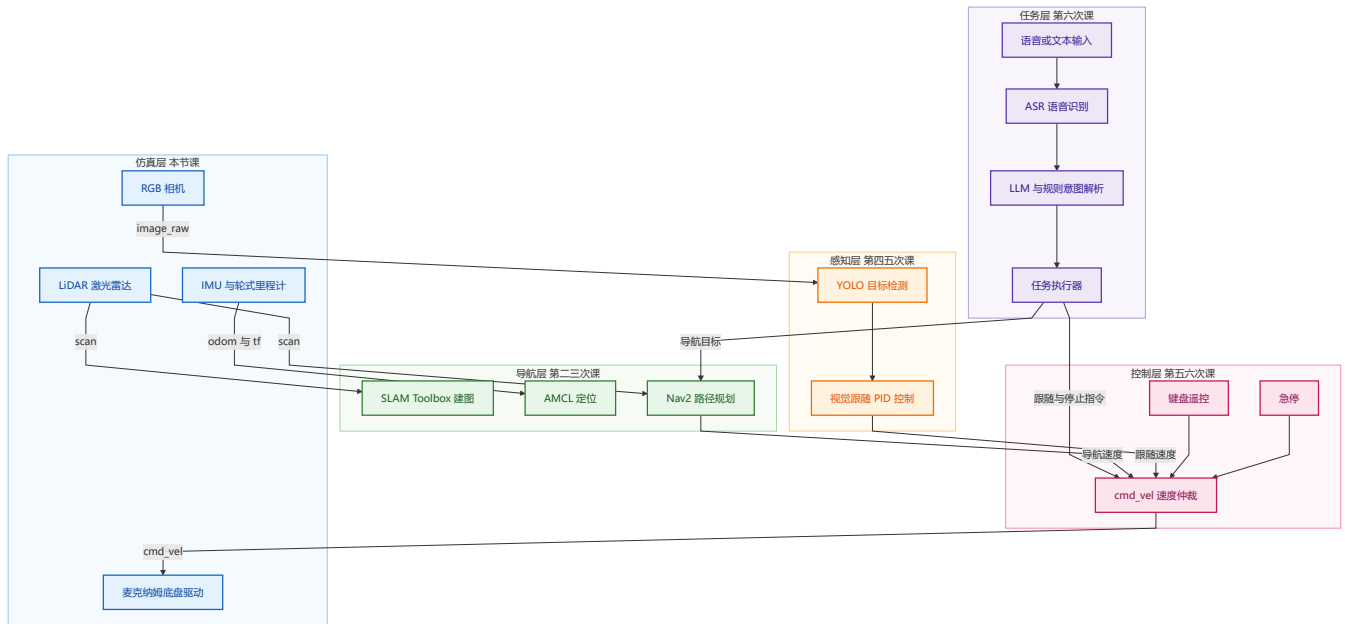
## 2.5 完成可观察、可验证的实验闭环

本节实验不以“窗口成功打开”为完成标准，而要依次验证：

1. 环境依赖完整，四个 ROS 2 包可以被正确构建和发现。
2. Gazebo 世界、机器人模型、物理系统和传感器正常加载。
3. ROS 2 能收到 LiDAR、IMU、Camera、里程计和 TF 数据。
4. `/cmd_vel` 能驱动机器人完成前后、横移和旋转。
5. GUI 不可用时，能够切换到 headless 或轻量仿真继续检查接口。
6. 自动验收脚本能够独立启动、测试并清理仿真进程。

## 3. 最终项目架构

六次课完成后的整体架构如下。本节课只搭建最下方的仿真层，后续每次课向上叠加一层：



数据从底层向上流动，控制命令从上层向下流动。例如 LiDAR 数据从仿真层发布到 `/scan`，SLAM 和 Nav2 订阅它；Nav2 计算出的速度通过 `/cmd_vel` 返回底盘。视觉跟随也会发布速度，所以后续必须增加速度仲裁，避免多个模块同时控制机器人。

本节只搭建最底层的仿真、传感器、里程计、TF 和速度控制。底层接口稳定后，后续模块才能逐层叠加。如果 `/scan`、`/odom`、`/tf` 的名称或坐标约定频繁变化，SLAM 和 Nav2 的配置也必须反复修改。

仿真与真机应尽量使用相同接口。例如算法层始终订阅 `/scan`，在 Gazebo 中由桥接节点提供，在真机中由激光雷达驱动提供。这样切换底层时，导航和感知代码不需要修改，这种设计称为接口解耦。

## 4. 本项目 ROS 2 包

```
src/
├─ mecamind_description # URDF/Xacro 与 RViz 机器人描述
├─ mecamind_gazebo      # Gazebo 模型、世界、桥接和启动文件
├─ mecamind_bringup     # 课堂统一入口
└─ mecamind_tools       # 轻量仿真及后续导航/交互工具
```

四个包的职责如下：

- `mecamind_description` 保存 URDF/Xacro 机器人描述，主要服务于 ROS 2 坐标树、RViz 和机器人外形展示。
- `mecamind_gazebo` 保存 Gazebo 使用的 SDF 模型、世界、传感器、驱动插件和桥接配置。
- `mecamind_bringup` 提供面向使用者的统一启动入口。上层使用者不必记住每个底层包中的启动文件。
- `mecamind_tools` 保存轻量仿真节点，以及后续课程使用的建图、导航、感知和任务工具。

主要文件：

```
mecamind_description/urdf/mecamind_mecanum.urdf.xacro
mecamind_gazebo/models/mecamind_mecanum/model.sdf
mecamind_gazebo/worlds/three_room_house.sdf
mecamind_gazebo/config/bridge.yaml
mecamind_gazebo/launch/gazebo_sim.launch.py
mecamind_bringup/launch/gazebo.launch.py
mecamind_bringup/launch/lite_sim.launch.py
```

执行 `colcon build` 时, `src/` 中的源码会被构建并安装到 `install/`。终端必须执行 `source install/setup.bash`, ROS 2 才能把当前工作空间加入包搜索路径。修改源码后如果忘记重新构建, 实际运行的可能仍是上一次安装版本。

## 5. 麦克纳姆轮运动学

### 5.1 机器人坐标系

机器人坐标系约定:

- `x`: 向前为正。
- `y`: 向左为正。
- `z`: 向上为正。
- `wz`: 逆时针旋转为正。

这些方向遵循 ROS 常用的右手坐标系。右手拇指指向 `z` 轴正方向, 四指弯曲方向就是绕 `z` 轴旋转的正方向。因此从机器人上方观察, `wz > 0` 表示逆时针旋转。

`/cmd_vel` 的消息类型为 `geometry_msgs/msg/Twist`, 常用字段为:

```
linear.x    前后线速度, 单位 m/s
linear.y    左右线速度, 单位 m/s
angular.z   绕竖直轴角速度, 单位 rad/s
```

对平面移动机器人而言, `linear.z`、`angular.x` 和 `angular.y` 通常保持为零。

### 5.2 从车体速度到车轮速度

设:

- 车体期望速度为 `vx`、`vy`、`wz`。
- 轮半径为 `r`。
- 前后轮中心距离的一半为 `Lx`。
- 左右轮中心距离的一半为 `Ly`。
- $K = Lx + Ly$ 。

一种常见的 X 型麦轮逆运动学为:

```

w_front_left  = (vx - vy - K*wz) / r
w_front_right = (vx + vy + K*wz) / r
w_rear_left   = (vx + vy - K*wz) / r
w_rear_right  = (vx - vy + K*wz) / r

```

公式中的加减号反映四个轮子的安装位置和滚子方向。例如只给出  $v_x$  时，四个轮子共同产生前后运动；只给出  $v_y$  时，四个轮子的转向组合产生横向运动；只给出  $w_z$  时，左右和前后车轮形成旋转力矩。

除以轮半径  $r$  是因为车体线速度需要换算成车轮角速度。轮半径越小，同样的车体速度需要越大的车轮角速度。 $K$  反映轮子到车体旋转中心的距离，轴距或轮距变化后，完成相同角速度所需的轮速也会变化。

这组公式描述的是**逆运动学**：已知车体希望怎样移动，计算每个轮子应该怎样转。反过来，根据四个轮子的实际速度估算车体速度，称为**正运动学**，里程计通常会用到它。

本项目机器人的实际参数在 `mecamind_gazebo/models/mecamind_mecanum/model.sdf` 中定义：

```

wheel_radius      = 0.075 m    即 r = 0.075
wheelbase         = 0.40 m     前后轮中心距离，即 Lx = 0.20
wheel_separation  = 0.34 m     左右轮中心距离，即 Ly = 0.17
K = Lx + Ly = 0.37

```

可以代入公式验证一次：设机器人只需要以  $v_y = 0.20 \text{ m/s}$  向左横移（ $v_x = 0$ 、 $w_z = 0$ ），则：

```

w_front_left  = (0 - 0.20 - 0) / 0.075 = -2.67 rad/s
w_front_right = (0 + 0.20 + 0) / 0.075 = +2.67 rad/s
w_rear_left   = (0 + 0.20 - 0) / 0.075 = +2.67 rad/s
w_rear_right  = (0 - 0.20 + 0) / 0.075 = -2.67 rad/s

```

四个轮子转速大小相同、方向两两相反：前左和后右反转，前右和后左正转。它们的前后分力互相抵消，横向分力叠加，机器人纯横移。课堂上可以让学员再代入  $w_z = 0.5 \text{ rad/s}$  的原地旋转情况，自己算一遍四个轮速。

不同厂商的轮子安装方向、关节正方向和电机正方向可能不同，因此工程中必须通过前进、横移、旋转和组合运动进行验证，不能只看公式。如果前进正确但横移方向错误，通常是滚子安装方向或某个轮子的符号配置错误。

Gazebo 中由 `gz::sim::systems::MecanumDrive` 根据 `/cmd_vel` 完成轮速分配。

## 6. 数据接口

### 6.1 话题与消息

本节使用以下话题：

|                                  |         |
|----------------------------------|---------|
| <code>/clock</code>              | 仿真时钟    |
| <code>/cmd_vel</code>            | 机器人速度指令 |
| <code>/odom</code>               | 轮式里程计   |
| <code>/scan</code>               | 二维激光雷达  |
| <code>/imu/data</code>           | IMU     |
| <code>/camera/image_raw</code>   | RGB 图像  |
| <code>/camera/camera_info</code> | 相机内参    |
| <code>/tf</code>                 | 动态坐标变换  |
| <code>/tf_static</code>          | 固定坐标变换  |

话题名称相同还不够，发布者和订阅者使用的消息类型也必须兼容。可以执行以下命令查看类型和连接数量：

```
ros2 topic type /cmd_vel
ros2 topic info /scan --verbose
ros2 interface show geometry_msgs/msg/Twist
```

`ros2 topic list` 只能证明话题已经被发现，不能证明它正在持续产生有效数据。检查传感器时还应使用 `ros2 topic echo --once` 查看内容，或使用 `ros2 topic hz` 检查发布频率。

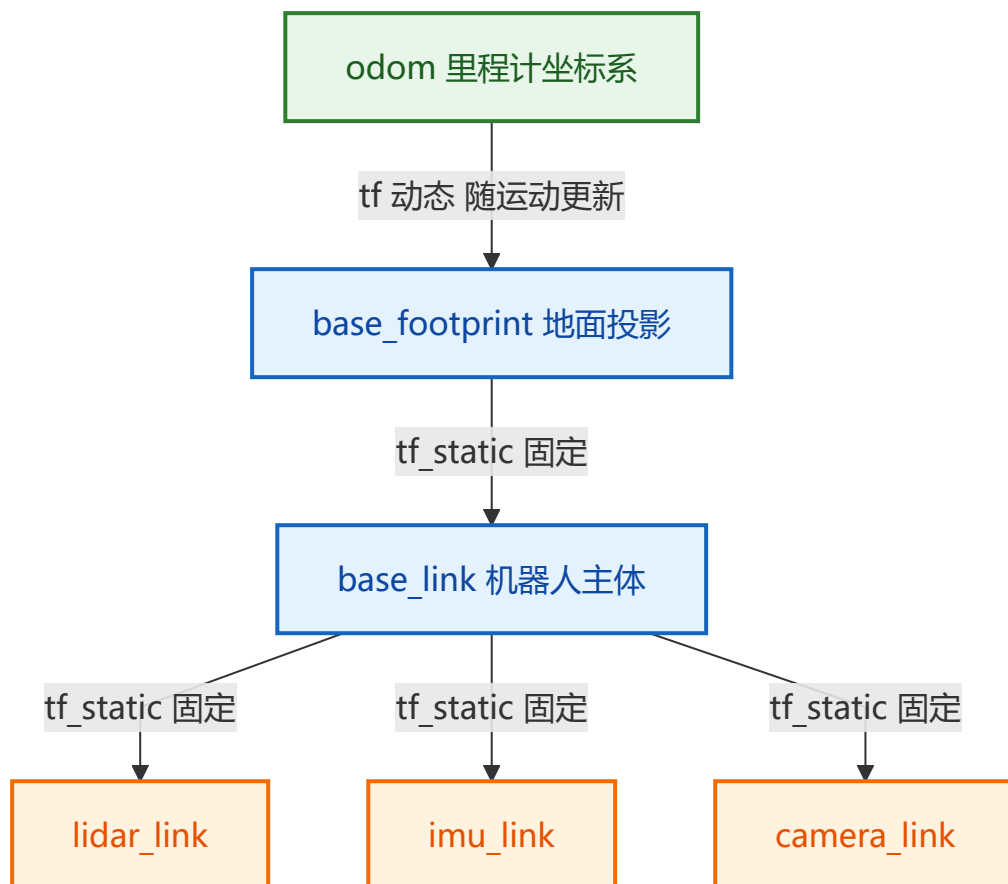
## 6.2 ROS 2 与 Gazebo 桥接

ROS 2 节点并不直接读取 Gazebo 内部消息。`ros_gz_bridge` 根据 `mecamind_gazebo/config/bridge.yaml` 在 ROS 2 消息与 Gazebo Transport 消息之间转换。

传感器数据主要从 Gazebo 流向 ROS 2，例如 `/scan`、`/imu/data` 和 `/camera/image_raw`；速度指令主要从 ROS 2 流向 Gazebo。排错时应分两侧检查：先用 `gz topic -l` 确认 Gazebo 端是否存在数据，再用 `ros2 topic list` 和 `ros2 topic echo` 检查 ROS 2 端。这样可以判断问题发生在传感器、桥接配置还是 ROS 2 节点。

## 6.3 TF 坐标变换

传感器测量值必须带有参考坐标系。例如激光点是在 `lidar_link` 下测得的，导航算法需要知道 `lidar_link` 相对机器人底盘和地图的位置。TF 使用一棵坐标树维护这些空间关系：



- `base_link` 表示机器人主体坐标系。
- `base_footprint` 是机器人在地面的投影，常用于二维导航。
- `odom` 是局部连续的里程计坐标系，短时间内平滑，但会逐渐累积漂移。

- 传感器坐标系描述传感器相对车体的安装位置和方向。

`/tf_static` 发布不会随时间变化的固定关系，例如 Camera 安装在车体前方；`/tf` 发布运行中变化的关系，例如机器人在 `odom` 中的位置。只看到传感器话题却没有正确 TF，SLAM 和 Nav2 仍然无法使用这些数据。

## 6.4 仿真时钟

Gazebo 发布 `/clock` 作为仿真时间。仿真可以暂停、加速或低于真实时间运行，因此仿真节点通常设置 `use_sim_time:=true`，让消息时间戳与 Gazebo 保持一致。如果部分节点使用系统时间、部分节点使用仿真时间，就可能出现“数据太旧”“无法进行坐标变换”等错误。

可以执行以下命令确认仿真时钟和节点参数：

```
ros2 topic echo --once /clock
ros2 param get /robot_state_publisher use_sim_time
```

## 7. LAB 1：环境检查

进入项目：

```
cd ~/mecamind_ros2
```

执行检查：

```
bash setup/check_environment.sh
```

正常结果最后一行：

```
MECAMIND_ENVIRONMENT_OK
```

如果缺少依赖：

```
bash setup/install_dependencies.sh
```

重新打开终端或执行：

```
source /opt/ros/jazzy/setup.bash
```

## 8. LAB 2：构建独立项目

```
cd ~/mecamind_ros2
bash scripts/build.sh
source install/setup.bash
```

检查包：



```
ros2 pkg prefix mecamind_description
ros2 pkg prefix mecamind_gazebo
ros2 pkg prefix mecamind_bringup
ros2 pkg prefix mecamind_tools
```

四条命令都应输出当前 `mecamind_ros2/install/` 下的安装路径。

## 9. LAB 3: 启动 Gazebo UI

```
source /opt/ros/jazzy/setup.bash
source ~/mecamind_ros2/install/setup.bash
ros2 launch mecamind_bringup gazebo.launch.py use_gui:=true
```

启动文件支持以下参数，均通过 `参数名:=值` 的形式附加在命令后：

| 参数                   | 默认值                            | 作用                                                  |
|----------------------|--------------------------------|-----------------------------------------------------|
| <code>use_gui</code> | <code>true</code>              | 是否打开 Gazebo 图形窗口； <code>false</code> 表示 headless 运行 |
| <code>world</code>   | 项目自带三房间世界                      | 指定要加载的世界 SDF 文件路径                                   |
| <code>display</code> | 自动检测，WSLg 下通常为 <code>:0</code> | Gazebo GUI 使用的 X11 显示地址                             |

预期看到：

- Gazebo 窗口正常打开。
- 三房间、走廊、桌子、柜体和柱子。
- 蓝色麦克纳姆机器人位于左下房间。
- 机器人顶部有红色 LiDAR，前部有 Camera。

不要急于遥控。先等待 5~10 秒，让 Gazebo Sensor 和 ROS-Gazebo Bridge 完成初始化。

结束仿真的正确方式：回到启动仿真的终端按一次 `Ctrl+C`，等待所有子进程打印退出信息并回到命令提示符，再关闭终端。直接叉掉终端窗口容易留下残留进程（处理方法见常见问题 14.7）。

## 10. LAB 4: 检查节点、话题和频率

新开终端（每个新终端都必须重新 `source` 这两个文件，否则找不到项目的包和话题）：

```
source /opt/ros/jazzy/setup.bash
source ~/mecamind_ros2/install/setup.bash

ros2 node list
ros2 topic list
```

逐项检查：

```
ros2 topic hz /scan
ros2 topic echo --once /imu/data
ros2 topic echo --once /odom
ros2 topic hz /camera/image_raw
ros2 topic echo --once /camera/camera_info
```

各传感器的预期发布频率由模型文件中的 `update_rate` 决定，`ros2 topic hz` 的输出应接近下表。仿真负载高时略低于标称值属于正常现象，但如果只有标称值的一半以下，应检查机器性能或渲染配置：

| 话题                             | 预期频率    | 来源                                                  |
|--------------------------------|---------|-----------------------------------------------------|
| <code>/scan</code>             | 约 10 Hz | LiDAR 传感器 <code>update_rate</code>                  |
| <code>/imu/data</code>         | 约 50 Hz | IMU 传感器 <code>update_rate</code>                    |
| <code>/camera/image_raw</code> | 约 15 Hz | Camera 传感器 <code>update_rate</code>                 |
| <code>/odom</code>             | 约 30 Hz | MecanumDrive 插件 <code>odom_publish_frequency</code> |

检查 TF 树：

```
ros2 run tf2_tools view_frames
```

该命令会监听几秒 `/tf` 和 `/tf_static`，然后在**当前目录**生成 `frames_日期_时间.pdf`。WSL 中可以执行 `explorer.exe`，打开当前目录，用 Windows 的 PDF 阅读器查看。图中应能看到 `odom` → `base_footprint` → `base_link` → 各传感器 的完整树。

图像可以用 `rqt_image_view` 查看：启动后在左上角下拉框选择 `/camera/image_raw`，应能看到机器人视角的彩色画面。

## 10.1 用 RViz 查看模型与坐标系

RViz 是 ROS 2 官方可视化工具，比静态 PDF 更直观。保持 Gazebo 运行，新开终端：

```
source /opt/ros/jazzy/setup.bash
source ~/mecamind_ros2/install/setup.bash
ros2 run rviz2 rviz2 --ros-args -p use_sim_time:=true
```

进入 RViz 后依次操作：

1. 左侧 `Global Options` 中把 `Fixed Frame` 改为 `odom`。
2. 点击左下 `Add`，选择 `RobotModel`，把它的 `Description Topic` 设为 `/robot_description`，应显示机器人三维模型。
3. 再 `Add` 一个 `TF`，可以看到所有坐标系的位置和朝向。
4. 再 `Add` 一个 `LaserScan`，`Topic` 设为 `/scan`，应看到激光点贴着墙体分布。

之后遥控机器人移动时，RViz 中的机器人和激光点会同步运动。这一步能直观确认 URDF、TF 和传感器数据三者是一致的。

## 11. LAB 5: 麦克纳姆轮遥控

启动键盘遥控：

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

推荐先把线速度设为约 0.2 m/s、角速度设为约 0.5 rad/s。

依次验证：

1. 小写 **i**：向前。
2. 小写 **o**：向后。
3. 小写 **j** / **l**：原地左转/右转。
4. 大写 **J** / **L**：向左/向右横移。
5. 大写 **I** / **<**：保持朝向前后平移。
6. 大写 **U**/**O**/**M**/**>**：组合平移。
7. **k**：停车。

注意终端大小写。横移使用大写按键，需要按住 Shift。

也可以直接发送可重复的速度命令。注意：执行前必须先在遥控终端按 **Ctrl+C** 退出 `teleop_twist_keyboard`。两个发布者同时向 `/cmd_vel` 发送不同速度时，命令会互相覆盖，机器人表现为抖动或时走时停：

```
# 前进
ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist \
  '{linear: {x: 0.20, y: 0.0}, angular: {z: 0.0}}'

# 左横移
ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist \
  '{linear: {x: 0.0, y: 0.20}, angular: {z: 0.0}}'

# 逆时针旋转
ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist \
  '{linear: {x: 0.0, y: 0.0}, angular: {z: 0.50}}'
```

按 **Ctrl+C** 停止持续发布后，再显式发送一次零速度：

```
ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist '{}'
```

## 12. LAB 6: 无界面与轻量兜底

Gazebo headless：

```
ros2 launch mecamind_bringup gazebo.launch.py use_gui:=false
```

headless 仍运行 Gazebo 物理和传感器，只是不打开图形窗口，适合自动测试。

如果 Gazebo 渲染或 GPU 驱动暂时不可用，可使用轻量仿真：

```
ros2 launch mecamind_bringup lite_sim.launch.py
```

轻量仿真保持 `/cmd_vel`、`/odom`、`/scan`、`/imu/data` 等核心接口，但不提供真实 Camera 图像和 Gazebo 物理效果。它是课堂兜底，不是 Gazebo 完成情况的替代证据。

## 13. CP1 自动验收

先确保项目已经构建，然后停止手工启动的仿真：

```
cd ~/mecamind_ros2
bash scripts/test_cp1.sh
```

脚本将：

1. 使用独立 ROS Domain 启动 headless Gazebo。
2. 检查 `/clock`、`/scan`、`/imu/data`、`/odom`。
3. 自动发送前进、横移和旋转指令。
4. 根据里程计验证三种运动。
5. 自动关闭本次启动的仿真。

成功标志：

```
MECAMIND_CP1_MOTION_OK
MECAMIND_CP1_OK
```

## 14. 常见问题

### 14.1 Gazebo 窗口不出现

检查：

```
echo "$DISPLAY"
echo "$WAYLAND_DISPLAY"
xdpyinfo -display :0
gz sim -v 4 shapes.sdf
```

WSLg 的标准 X11 地址通常是 `:0`。如果当前 `DISPLAY` 指向类似 `172.x.x.x:0` 且无法连接，而 `xdpyinfo -display :0` 成功，可以执行：

```
export DISPLAY=:0
```

项目 launch 检测到 `/tmp/.x11-unix/x0` 时也会自动给 Gazebo 子进程使用 `:0`。如果两个显示变量都为空，先修复 WSL GUI；上课可临时使用 `use_gui:=false`，但课后仍需完成 UI 验收。

## 14.2 Ogre2、OpenGL 或显卡错误

先确认 Windows 显卡驱动和 WSL 更新：

```
ws1 --update  
ws1 --shutdown
```

如硬件加速异常，可临时测试软件渲染：

```
export LIBGL_ALWAYS_SOFTWARE=1  
ros2 launch mecanind_bringup gazebo.launch.py use_gui:=true
```

软件渲染较慢，只用于定位问题。

## 14.3 Gazebo 有机器人，但 ROS 2 没有话题

检查桥节点：

```
ros2 node list | grep bridge  
ros2 param get /mecamind_gz_bridge config_file  
gz topic -l
```

若 Gazebo 有 `/scan` 而 ROS 没有 `/scan`，重点检查 `bridge.yaml` 的消息类型和方向。

## 14.4 有 `/cmd_vel`，机器人不动

检查命令：

```
ros2 topic echo /cmd_vel  
gz topic -e -t /cmd_vel
```

如果 ROS 有而 Gazebo 没有，问题在桥接；如果 Gazebo 也有，检查 MecanumDrive 插件、关节名和世界日志。

## 14.5 能前进和旋转，但不能横移

优先检查：

- 是否使用大写 `J/L`。
- `Twist.linear.y` 是否非零。
- 四个麦轮安装方向和摩擦方向是否成对相反。
- MecanumDrive 的前后左右关节是否配置正确。

## 14.6 没有 Camera 图像

Camera 依赖 Gazebo Sensors System 和渲染引擎。检查：

```
gz topic -l | grep camera  
ros2 topic list | grep camera
```

headless 下仍需可用的渲染后端；若 WSL 环境暂时无法建立渲染上下文，本节其他传感器和运动仍可通过轻量仿真继续，Camera 问题必须在第四次课前解决。

## 14.7 多个仿真同时运行导致话题混乱

正常情况下，在启动仿真的终端按 `Ctrl+C`，等待所有子进程退出。若终端被直接关闭或测试异常中断，可以先预览并清理本项目残留：

```
cd ~/mecamind_ros2
bash scripts/cleanup_ros2.sh --dry-run
bash scripts/cleanup_ros2.sh
```

如果确实需要同时运行多个实验，应为每套实验设置不同 Domain：

```
export ROS_DOMAIN_ID=81
```

不同 Domain 的节点不会互相发现，但它不能解决同一 Gazebo 资源、显示设备或文件被重复占用的问题。不要用全局 `pkill` 作为日常操作，以免结束其他项目的 ROS 2 进程。只有确定当前用户的全部 ROS 2 和 Gazebo 进程都可以终止时，才使用 `bash scripts/cleanup_ros2.sh --all`。

## 15. 本节验收清单

- ☐ 能说明项目五层架构和六次课迭代关系。
- ☐ 环境检查输出 `MECAMIND_ENVIRONMENT_OK`。
- ☐ 四个 ROS 2 包成功构建。
- ☐ Gazebo UI 能显示三房间和麦轮机器人。
- ☐ `/scan`、`/imu/data`、`/camera/image_raw`、`/odom` 有数据且频率接近预期值。
- ☐ TF 中包含 `odom`、`base_footprint`、`base_link` 和传感器坐标系。
- ☐ RViz 中能看到机器人模型、TF 坐标系和激光点，且随遥控同步运动。
- ☐ 完成前进、后退、横移和旋转。
- ☐ 能启动 headless Gazebo。
- ☐ 能启动轻量仿真兜底。
- ☐ `scripts/test_cp1.sh` 输出 `MECAMIND_CP1_OK`。

## 16. 课后作业

必做：

1. 保存 Gazebo UI 截图。
2. 保存 `/scan`、`/imu/data`、`/odom` 的一次输出。
3. 录制不超过 30 秒的前进、横移和旋转视频。
4. 画出本机实际运行的节点—话题图。可以先运行 `ros2 run rqt_graph rqt_graph` 查看实际连接关系，再照着手绘或截图标注。

选做（均需修改后重新执行 `bash scripts/build.sh` 并重启仿真才会生效）：

1. 修改机器人车身颜色：编辑 `src/mecamind_gazebo/models/mecamind_mecanum/model.sdf` 中车身 Link 的 `material` 颜色值。
2. 修改 Camera 分辨率：编辑同一文件中 Camera 传感器的 `width` 和 `height`，用 `ros2 topic bw /camera/image_raw` 比较修改前后的话题带宽。
3. 调整轮距、轴距或轮半径：修改 `MecanumDrive` 插件的 `wheel_separation`、`wheelbase` 或 `wheel_radius`，让参数与实际模型不一致，观察错误参数对 `/odom` 的影响。
4. 在 world 中增加一个静态障碍物：编辑 `src/mecamind_gazebo/worlds/three_room_house.sdf`，参考已有的柱子添加一个 box 模型，用 RViz 的 LaserScan 确认 LiDAR 能检测到它。